

Dev Team Report: End-to-End Model/Checkpoint/Docs Audit + Smoke-Test Harness (DeepMamba2 / CondorBrain™)

Why this exists

If the .pth checkpoint doesn't encode what we think it encodes (feature order, head mapping, targets, normalization, loss composition, learned conditions, etc), **every downstream trade decision is suspect**. That mismatch is a top reason when we first met I stated *"few models work, most fail live"*: live execution magnifies any silent I/O contract drift from backtesters – In bsccktesting, even with very high up to 100%-win rates, 99.99% of the time once the automated trading system is placed on the live markets, it fails miserably. When tested on a practice account, > 90% still fail on the live market accounts.

As discouraging as it seems, as if it's damn near next to impossible to be able to test if an automated trading strategy is going to work or not, until we are risking/losing real money in a real live trading account. For most, yes, this would be the case, and it's unfortunate but it's how the banks maintain their dominance over the markets, and why so many retail traders might make a little money, then blink an eye, and they end up losing their hard-earned money. For this reason, we absolutely must do forensic level due diligence to ensure that the trained model file takes live accounts to the promised land. But that is only one part of the "pre-check" list.

To ensure an account passes the demo account stage and has a chance at working for real on a real live trading account, we have to take into consideration:

a) **market data manipulation** – market makers will set up massive orders on the ladder outside of the scope of retail traders level 2 data they can see. This causes huge gaps to occur, often leading to a stop-out or margin call, and loss of money. Slippage is another "excuse" the brokers attempt to get away with and do so for 99.99% of traders globally – hence the "friction gate" trading rule I implemented.

b) **trade data manipulation** – this is very illegal to do, but it happens often, and most retail traders don't even realize the broker has just screwed them over and taken their hard-earned money. Try and have their team audit the trade(s)? They will come back every time and say, *"We are sorry, but we saw no issues with your position you opened and closed (automated, manually, or by force – margin call)"*. Where does that leave the retail traders – SOL. Where does that leave the liquidity pools and banks? With your money. Luckily, I know absolutely ever single tactic they use, and have recordings of many of them on video, and believe me, if you don't know what to look for, you would never even catch it, and hence why I am kicked out of every trading firm and why the banks hate me and one of the reasons I am currently homeless. When you gather hard evidence to bust their crooked asses, you end up in my position, with your name heavily slandered in the press globally, your best friends betraying you, and end up on the streets.

So, then how do we get around these big bank/liquidity pool tactics? You have someone with my know-how and knowledge which there are probably < 100 people globally that actually fully understand what happens and precisely where to look and when to look for it, and we compel using additional checks and balances to the trade rules and bake them into the AI-NN models.

The key is: **you can massively improve the odds before risking real capital** by combining:

1. **Audit** - The full model/contract audit plan I have built and currently going through the forensic level steps currently on the Quantor-MTFuzz™ repo, **plus**
2. **Keep the brokers real database** - live microstructure gates + execution integrity monitoring, **plus**
3. **Integrate Banking Tactics** - a strict demo → *live promotion process** with kill switches and tight limits.

That doesn't *guarantee profits*, but it turns "unknown failure" into "controlled, measurable risk" and prevents the common catastrophic paths. Remember what we discussed the other day when I stated, "*A good trader is not one who knows how to make profits on trades, anyone can get lucky, a good trader knows how to handle, minimize and in a disciplined manner, make up for loss, as that happens on every account*"

** - See Appendix at end of this DevOps report/update for a full explanation of how we get around the big bank/liquidity pool brick wall.*



End-to-End Model / Checkpoint / Backtest / Live Audit ("Pen-Test Harness")

Version: 1.1 (DevOps-Expanded)

Author: Dr. T. Jerry Mahabub, Ph.D.

Date: 2026-01-26

Owner: Engineering / Quant Research + DevOps (CI/CD + Release Governance)

0) What DevOps is being asked to own (in one sentence)

DevOps will **turn model correctness into a release gate** by enforcing a strict audit harness that prevents **silent contract drift** between training, checkpoint serialization, backtesting, and live execution.

1) Why this is critical (plain English)

Live markets expose every small mismatch in:

- **inputs** (feature order, missing columns, normalization drift),
- **outputs** (head mapping drift, thresholding drift),
- **execution** (fills, multi-leg partial fills),
- **timing** (session boundaries, DST, stale chain snapshots).

A model can "look fine" in a notebook yet fail live if any of the following happens:

- the live feature builder produces a slightly different vector than the trainer used,
- the `.pth` has a different head layout than the live code assumes,
- normalization stats are missing or applied differently,
- backtest interprets logits differently than live,
- stale market data causes entries at the wrong time,
- multi-leg spreads partially fill and the system doesn't handle it safely.

This harness makes those failure modes impossible to ship. It treats the entire model pipeline like a security boundary and "pen-tests" every contract.

2) DevOps deliverables (what must exist in the repo + CI/CD)

2.1 CI/CD pipeline gates (must be enforced)

DevOps must implement **mandatory checks** on PRs and on release candidates:

- Run smoke suite on every PR that touches model/data/strategy code
- Block merge on failures
- Produce audit artifacts (manifest, contract snapshot, trace + attribution) for release candidates
- Store artifacts with hashes for reproducibility
- Provide a deterministic “golden fixture” test path to catch regressions

2.2 Artifact management & trace retention

DevOps must ensure artifacts land in:

- `artifacts/audit/` (manifests, contract snapshots, DAG PDFs, audit packet PDFs)
- `artifacts/learned_conditions/` (decision traces, attributions, surrogate rules)

...and are retained per release policy (e.g., last N releases + last N training runs, or S3/Blob equivalent).

3) The core idea: “Contracts” (what we freeze and enforce)

A **contract** is a strict agreement between components. The model only works if all contracts hold.

Contract 1 — Feature Vector Contract

What it covers: ensures the model sees the same feature vector in training/backtest/live.

Why it matters: if feature order changes or a column is missing, the model’s learned weights map to the wrong signals.

How we enforce: feature schema ID = hash of feature list; parity tests; strict schema validation.

Contract 2 — Target/Label Contract

What it covers: ensures the model is trained on the exact target that the system believes it’s predicting.

Why it matters: if targets differ (horizon/threshold), “good training” won’t translate to live decisions.

How we enforce: target definitions are versioned; unit tests on synthetic data; leakage tests.

Contract 3 — Head Mapping Contract

What it covers: defines what each model output means (entry, exit, sizing, diffusion, etc.).

Why it matters: a head index swap or shape change makes live trade decisions nonsensical.

How we enforce: outputs are named (not inferred by index), forward contract tests, checkpoint metadata checks.

Contract 4 — Checkpoint Contract

What it covers: ensures .pth includes enough metadata to interpret it correctly.

Why it matters: missing metadata forces guessing; guessing breaks live.

How we enforce: required keys + hashes must exist; CI fails if missing.

Contract 5 — Interpretation Contract

What it covers: ensures backtest and live interpret outputs identically.

Why it matters: if backtest uses sigmoid and live uses raw logits, results will diverge.

How we enforce: shared decision-mapper module + parity SIM test.

Contract 6 — Docs Contract

What it covers: prevents docs from being out-of-sync with actual code behavior.

Why it matters: teams ship the wrong assumptions.

How we enforce: docs contract file must match code-derived contract; CI fails on drift.

Contract 7 — Time & Session Contract

What it covers: session boundaries, DST, holidays, bar alignment, spot↔options time alignment.

Why it matters: off-by-one bars and time skew cause systematic bad entries/exits live.

How we enforce: alignment tests on fixtures; explicit session rules.

Contract 8 — Execution & PnL Accounting Contract

What it covers: fees/slippage/multipliers/rounding, and how trades are priced and accounted.

Why it matters: you can “match decisions” yet still diverge on real PnL due to accounting mismatch.

How we enforce: parity tests on a fixed trade log; tolerances defined.

Contract 9 — Risk Gate Precedence Contract

What it covers: which gates run first and what “blocked” means operationally.

Why it matters: wrong precedence can cause live to trade when it should halt.

How we enforce: precedence matrix documented + tested.

Contract 10 — Determinism & Reproducibility Contract

What it covers: run manifests capture environment, seeds, versions, and determinism settings.

Why it matters: without this, you cannot reproduce “why model X failed live.”

How we enforce: run manifest must be produced; CI checks presence.

Contract 11 — Data Freshness & Staleness Contract

What it covers: detects stale bars/chains/greeks and forces safe behavior.

Why it matters: stale data is a top live failure cause.

How we enforce: staleness tests; runtime gate behavior: **no-entry / exit-only**.

Contract 12 — Instrument & Contract Selection Contract

What it covers: OCC parsing, strike rounding, expiry selection, DTE filters.

Why it matters: wrong legs = wrong trade = wrong risk.

How we enforce: OCC roundtrip tests; expiry/calendar tests.

Contract 13 — Multi-leg Execution Integrity Contract

What it covers: partial fills, cancel/replace, leg imbalance safety.

Why it matters: spreads fill unevenly live; must avoid naked leg exposure.

How we enforce: state machine tests; simulated partial fill scenarios.

Contract 14 — Safety Controls / Kill Switch Contract

What it covers: halting conditions (loss limits, anomalies, staleness, collapse).

Why it matters: this prevents account blowups and runaway trading.

How we enforce: explicit trigger tests; required logs.

Contract 15 — Backward Compatibility Contract

What it covers: ensures old checkpoints cannot be loaded under incompatible registries or head maps.

Why it matters: “wrong model loaded” is a frequent live incident.

How we enforce: checkpoint version compatibility tests; hard reject incompatible.

4) The test suite (what each script does and why DevOps should care)

All tests live under `scripts/model_tests/`. DevOps gates merges/releases on these.

01_checkpoint_inventory.py

Covers: Checkpoint Contract (4), Backward Compatibility (15)

Checks: checkpoint keys exist; sizes; metadata present; version fields.

Why: prevents “loading a checkpoint that cannot be interpreted correctly.”

02_feature_schema_alignment.py

Covers: Feature Vector Contract (1)

Checks: feature list in registry matches checkpoint; ordering hash matches.

Why: prevents silent feature order drift.

03_norm_stats_sanity.py

Covers: Feature Vector Contract (1), Checkpoint (4)

Checks: median/mad shapes match; finite values; protection rules applied correctly.

Why: normalization drift breaks live decisions.

04_model_forward_contract.py

Covers: Head Mapping (3), Interpretation (5)

Checks: forward outputs exist; shapes; named keys; finite values.

Why: ensures the model’s outputs can be safely interpreted.

05_output_distribution.py (Collapse Guard)

Covers: Collapse prevention; live safety

Checks: output variance, entropy, action rate sanity; detect constant outputs.

Why: prevents shipping “mean guesser” models that look “loaded” but produce nonsense.

06_ruleset_execution_smoke.py

Covers: Risk Gate Precedence (9), Interpretation (5)

Checks: rules load; required columns exist; “blocked” semantics present.

Why: rule engine failures can force trades or block exits.

07_feature_pipeline_parity.py

Covers: Feature Vector (1), Target/Label (2) indirectly

Checks: train vs backtest feature pipeline parity; NaN fill policy parity.

Why: pipeline drift makes backtests meaningless.

08_live_trading_parity_sim.py (Most important for live)

Covers: Interpretation Contract (5), Time/Session (7), Risk gates (9)

Checks: for the same slice, backtest interpreter and live interpreter generate identical actions and sizing.

Why: this catches the majority of “works in backtest, fails live” issues.

09_checkpoint_diff_epochs.py

Covers: Checkpoint (4), Backward Compatibility (15)

Checks: differences between e2 vs e3; head shapes, feature list, normalization changes.

Why: prevents accidentally deploying a materially different checkpoint.

10_decision_trace_schema_validate.py

Covers: Governance + accountability

Checks: decision_trace.jsonl validates; required provenance fields present.

Why: without trace integrity, you cannot audit incidents.

11_factor_attribution_aggregate.py

Covers: Explainability + performance attribution

Checks: builds per-factor wins/losses scoped to ENTRY/EXIT/SIZE.

Why: prevents misleading conclusions about what “works.”

12_docs_contract_sync_check.py

Covers: Docs Contract (6)

Checks: docs contract matches code contract; CI fails on drift.

Why: reduces operational errors caused by stale docs.

test_time_alignment_and_session.py

Covers: Time & Session (7)

Checks: bar alignment; DST/holiday logic; off-by-one detection.

Why: time bugs are catastrophic live.

test_pnl_accounting_parity.py

Covers: Execution & PnL (8)

Checks: given same trades/prices, backtest and live accounting match.

Why: prevents false performance attribution.

test_model_autodiscovery_selection.py

Covers: Backward compatibility + deployment safety

Checks: auto-discovery selects the intended checkpoint.

Why: prevents “wrong model loaded” incidents.

test_data_staleness_gate.py

Covers: Data freshness (11)

Checks: stale inputs block entry and permit exit-only; logging required.

Why: stale data is a frequent live failure cause.

test_occ_symbol_roundtrip.py

Covers: Instrument selection (12)

Checks: OCC parse → reconstruct yields identical contract.

Why: prevents wrong strikes/expiries/rights.

test_expiry_calendar_and_dte.py

Covers: Instrument selection + time rules (12, 7)

Checks: DTE computation, holiday impacts, weekly/monthly selection rules.

Why: expiry mis-selection breaks risk.

test_multileg_fill_state_machine.py

Covers: Multi-leg execution integrity (13)

Checks: partial fill sequences result in safe terminal state (no naked exposure).

Why: this is core for spread trading reliability.

test_kill_switch_triggers.py

Covers: Kill switch (14)

Checks: triggers fire under configured conditions; system halts and logs.

Why: safety net for live operations.

test_checkpoint_version_compat.py

Covers: Compatibility (15)

Checks: incompatible checkpoint is rejected with clear error.

Why: prevents deploy-time surprises.

5) DevOps workflow integration (what runs when)

5.1 PR Gate (fast)

Run a subset (time-bound):

- 01, 02, 03, 04, 05, 06, 07, 12
Fail fast on contract drift or collapse.

5.2 Release Candidate Gate (full)

Run everything:

- entire suite including parity SIM (08), staleness, multi-leg, kill switch
Generate:
 - run_manifest.json
 - contract_snapshot.json
 - decision_trace.jsonl (from a fixture run)
 - decision_factor_attribution.csv
 - audit packet PDF

5.3 Nightly (optional, recommended)

- checkpoint diffs across recent epochs
 - longer parity SIM slices across multiple volatility regimes
 - regression backtests on golden fixtures
-

6) Artifact generation (what DevOps must ensure is produced and stored)

Required artifacts per RC model

- artifacts/audit/run_manifest.json
Why: reproducibility (commit, env, dataset hash, checkpoint hash).

- `artifacts/audit/contract_snapshot.json`
Why: the single authoritative contract state for that model build.
- `artifacts/learned_conditions/decision_trace.jsonl`
Why: per-decision provenance and auditability.
- `artifacts/learned_conditions/decision_factor_attribution.csv`
Why: outcome attribution by factor per scope.
- `artifacts/audit/Accountability_Audit_Packet.pdf`
Why: governance-ready bundle.

Storage + retention recommendation

- Store artifacts keyed by `(model_hash, run_id, git_commit, dataset_hash)`
- Keep at least:
 - last N release candidates
 - last N production models
 - all artifacts for any model that traded live

7) Release acceptance criteria (DevOps “Go/No-Go” checklist)

A model can proceed to demo/live only if:

-  All smoke tests pass
-  Collapse guard passes
-  Parity SIM passes (backtest interpreter == live interpreter)
-  Time/session alignment passes
-  Execution/PnL parity passes
-  Staleness gates pass (stale → no-entry / exit-only)
-  Kill switch triggers pass
-  Multi-leg state machine pass
-  Docs contract sync passes
-  All required artifacts generated + stored

8) Why this reduces GPU waste + speeds iteration

GPU training is expensive. Without these gates:

- you spend GPU cycles producing models that can't run correctly live,
- you can't determine whether failure is due to training, I/O drift, or execution drift.

With these gates:

- failures are caught immediately,
 - fixes are precise,
 - GPU spend is applied only after the pipeline is proven correct.
-

9) Immediate DevOps action list (ordered)

1. Add CI job(s) to run PR smoke subset.
 2. Add RC pipeline stage to run full suite + produce artifacts.
 3. Add artifact upload/retention policy (even if local first).
 4. Add docs contract sync gate (`12_docs_contract_sync_check.py`).
 5. Add “one shared DecisionMapper module” requirement (backtest/live import same code).
 6. Add alerting hooks for staleness gates and kill switch triggers (live runtime).
-

10) Notes on “what caused live failures historically”

Based on observed symptoms (collapse, constant confidence, huge number of identical signals), the system likely suffered from:

- posterior collapse (constant outputs) **and/or**
- interpretation mismatch (thresholding or head mapping) **and/or**
- feature order drift between registry and runtime.

This harness directly targets those failure modes.

Appendix: Live-Execution Reality Controls (Beyond Model Contract Integrity)

Why this section exists

Even if the .pth contract is perfect, live performance can still fail because **live execution is a different physics environment** than backtesting:

- spreads widen or gap
- liquidity disappears
- order books shift faster than retail feeds
- fills happen at worse prices than mid
- partial fills create unintended exposure
- latency creates adverse selection

The goal is to stop treating those as “mystery losses” and instead produce **measurable, auditable signals** that:

1. prevent bad entries,
2. reduce slippage risk, and
3. generate a dispute-quality evidence trail when execution deviates materially from expectation.

A) Live “Friction & Integrity” Gate (Pre-Trade + In-Trade)

What it covers

This is the runtime layer that decides: **is it safe to enter/adjust right now** given current market conditions?

What it measures (all objective)

1. Spread Gate

- $\text{spread} = \text{ask} - \text{bid}$
- reject entries if spread exceeds a policy threshold (absolute or % of price)

2. Liquidity Gate

- minimum volume / minimum NBBO size (when available)
- reject entries when liquidity is thin

3. Volatility / Gap Gate

- detect abnormal bar ranges vs ATR
- block entries when volatility regime flips suddenly

4. Staleness Gate

- block entry if quote/bar/chain timestamps are stale
- allow exit-only mode when stale

5. Latency Gate

- measure order ack latency and market-to-fill latency
- block if latency crosses a threshold (adverse selection risk)

6. Slippage Guard (Pre-Trade)

- predicted slippage budget
- reject if expected slippage exceeds edge

Why it matters

Most live blowups happen when the system trades through poor microstructure conditions. These gates keep the system from being “*right but unfillable*.”

What DevOps needs to enforce

- Every gate must emit a **reason code** and be logged to `decision_trace.jsonl`
- Policy thresholds live in a versioned config: `docs/PIPELINE_CONTRACT.json` → `contract_snapshot.json`

B) Execution Integrity Monitoring (Post-Trade Verification)

What it covers

This verifies that the broker/exchange execution we got matches what we expected **within tolerance**.

What it measures (objective comparisons)

1. Expected vs Actual Fill Price

- baseline reference: bid/ask at decision time or order-submit time
- compute `slippage = fill_price - reference_price` (signed)

2. NBBO Deviation Evidence

- record NBBO snapshot at:
 - decision time
 - order submit
 - order ack
 - fill
- quantify if fills are consistently outside expected microstructure bounds

3. Partial Fill State Machine

- ensure multi-leg strategies never leave unintended exposure
- if partial fill occurs:
 - hedge/flatten policy triggers
 - cancel remaining legs
 - log event

4. Order Lifecycle Integrity

- track order id, replace/cancel chain, reject reasons
- reconcile “requested” vs “executed” actions in `decision_trace.jsonl`

Why it matters

It turns “the broker screwed me” from a feeling into a measurable anomaly report:

- slippage distribution
- latency spikes
- reject clusters
- abnormal partial fill patterns

DevOps deliverable

- a daily “execution integrity report” artifact:
 - `artifacts/audit/execution_integrity_daily.json`
 - `artifacts/audit/execution_integrity_daily.pdf` (optional)

C) “Dispute Packet” Evidence Bundle (When Something Looks Wrong)

What it covers

If you ever need to challenge a bad fill or explain unexpected losses, we need an evidence bundle that is generated into a pdf full-report with forensic level detail:

- complete
- timestamped
- hash-verifiable
- easy for a third party to review

What it contains

1. `run_manifest.json` (commit, model hash, dataset hash, env versions)
2. `contract_snapshot.json` (feature schema id, head mapping, thresholds, gate precedence)
3. `decision_trace.jsonl` (decision → action → execution → outcome)
4. NBBO snapshots around the trade lifecycle
5. Broker execution reports / order events (where available)
6. Slippage + latency calculations for the events including snapshots of entry and exit
7. A summary PDF that explains the anomaly in plain language (auto-generated)

Why it matters

This is the “*forensic level due diligence*” describing but grounded in objective artifacts.

D) Add These Tests to the Harness (DevOps gating)

We already added staleness, multi-leg, kill switch, accounting parity. Add these specifically:

1) `test_spread_and_liquidity_gates.py`

Ensures: entries are blocked when spreads are wide / liquidity is thin.

2) `test_slippage_budget_enforcement.py`

Ensures: if expected slippage exceeds edge, the trade is rejected.

3) `test_execution_anomaly_detector.py`

Ensures: repeated outlier slippage/latency triggers kill switch / risk-off.

These tests simply make the system **robust to harsh execution conditions**.